ФСП Карьерный профиль

Практическое руководство по локальному API

11 октября 2026 года

88 путей / 101 операция. Описывается API проекта, включая отдельный режим показа жюри. Все демонстрационные данные синтетические.

Версия сервиса: **0.4.0**, сверка 11 октября 2026. Рабочая копия NAS: /Volume1/projects/Fsp. Это API конкурсного продукта на синтетических данных. В контракте **88 путей и 101 операция**: несколько путей имеют разные методы. Официальный API ФСП этим документом не описывается.

Где находится контракт

- Базовый адрес на NAS: http://127.0.0.1:8470.
- Читаемая страница локального API: /api/docs.
- Машинный контракт: /api/openapi.json, сохранённая копия — contracts/openapi.yaml (../contracts/openapi.yaml).
- Все точные поля, обязательность, значения по умолчанию, enum и пределы — components.schemas этого контракта. В справочнике ниже приведены имена схем.
- Готовые запросы с placeholders — api-examples.http (api-examples.http). Они выполняются по отдельности: сначала вход, затем подстановка полученных ID и revision. Автоматическое выполнение всего файла не предполагается.

Открытие с другого компьютера через согласованный SSH-туннель и Docker-команды запуска описаны в README (../README.md) и актуальное руководство (https://github.com/punkpsyh-hash/Fsp/blob/main/docs/44-deployment-and-operations.md). Заголовок Host допускает 127.0.0.1, localhost, внутренний app и тестовый testserver. Прямое обращение по произвольному hostname не настроено.

Сессия, роли и безопасные примеры

Пользовательская авторизация — cookie fsp_session, HttpOnly, SameSite=Strict, Path=/api, срок 8 часов. Вход и подтверждение возвращают SessionView: id, role, label, csrf, demo; /api/v1/me дополнительно возвращает email. Cookie передаётся клиентом, токен csrf — заголовком X-CSRF-Token для пользовательских POST/PUT/PATCH/DELETE, включая logout. GET не требует CSRF. Получить текущий CSRF можно через /api/v1/me.

Кандидат и работодатель используют отдельные сессии. Роль проверяется сервером; ID в теле/URL не позволяет читать чужую попытку,

банк, подборку или приглашение. Чужая и отсутствующая запись обычно дают одинаковый 404. Общие /me и logout доступны обеим ролям. Публичные auth-операции не требуют уже созданной сессии. Внутренний runner имеет отдельную Bearer-авторизацию, описанную ниже.

Изменяющий запрос с Origin принимается только от локальных origins http://127.0.0.1 или http://localhost на портах 8470, 18080, 5173. Запрос curl без Origin допустим. CORS для произвольного внешнего сайта нет. Все POST /api/v1/auth/* ограничены 30 запросами за 60 секунд с одного IP в текущем процессе; превышение — 429 и Retry-After: 60.

Для первого просмотра достаточно публичного GET и входа в существующий seed:

Пример · bash

```
curl --fail-with-body http://127.0.0.1:8470/api/health
curl --fail-with-body http://127.0.0.1:8470/api/v1/auth/demo
-accounts
# Cookie-файл хранится локально вне проекта. Не коммитьте ег
o или ответ входа.
umask 077
curl --fail-with-body -c /tmp/fsp-candidate.cookies -o /tmp/
fsp-candidate-login.json \
  -H 'Content-Type: application/json' \
  --data '{"user_id":"demo-candidate-a"}' \
  http://127.0.0.1:8470/api/v1/auth/demo-login
curl --fail-with-body -b /tmp/fsp-candidate.cookies \
  http://127.0.0.1:8470/api/v1/candidate/profile
```

Ответ входа с CSRF сохраняется в локальный JSON с закрытыми правами, а не выводится в терминал. Cookie и этот JSON не коммитятся; удалите их после проверки.

Следующий запрос использует CSRF из собственного ответа входа через локальную переменную FSP_CANDIDATE_CSRF; его значение не записывается в документацию:

Пример · bash

```
curl --fail-with-body -b /tmp/fsp-candidate.cookies \
  -H "X-CSRF-Token: ${FSP_CANDIDATE_CSRF}" \
  -X DELETE http://127.0.0.1:8470/api/v1/candidate/fsp-link
```

Индивидуальный вход на стенд жюри

Только при FSP_JURY_MODE=1 и FSP_JURY_INDIVIDUAL_REGISTRATION=1 доступен POST /auth/jury-register. Тело JuryRegister содержит role (candidate или employer), password от 12 до 128 символов и consent: true. Другие поля, включая настоящую почту и имя, отклоняются. Сервер создаёт синтетический логин jury-…@example.invalid, профиль или компанию и сразу возвращает JurySession со статусом 201: id, role, label, email, csrf, demo.

Сохраните логин; после выхода используйте /auth/password-login. Пароль не возвращается сервером и не сохраняется в браузере. Подтверждение реальной почты этот режим не выполняет; полный локальный контур подтверждения описан ниже. Регистрацию ограничивают 200 аккаунтов и пять запросов в час на клиентский IP. Переполнение и частые запросы возвращают 429, отсутствие opt-in — 404. Посторонний Origin отклоняется до расходования лимита. Запрос из действующей сессии возвращает 409: сначала нужно выйти. Общие seed-аккаунты и mock-почта в индивидуальном показе недоступны. Последующие изменения требуют CSRF.

Регистрация и профиль

POST /api/v1/auth/register принимает email, role, label, consent:true и необязательный password длиной 12–128 символов. Разрешены только синтетические адреса name@example.invalid. При успехе 201 возвращает DeliveryView: message, transport (local-mailbox/smtp), delivery (ready/failed), demo:true. Сбой SMTP не отменяет регистрацию: ссылка уже подготовлена, доставку можно повторить через /auth/login.

GET /auth/mock-mail открыт и содержит одноразовые токены исключительно локальной демонстрационной почты. Срок ссылки — 30 минут. /auth/verify принимает { "token": "<полученный локально токен>" }, подтверждает адрес и создаёт сессию; повторное/просроченное подтверждение — 409. /auth/password-login принимает email/password, требует подтверждённого адреса; неправильные данные — 401,

неподтверждённый адрес — 403. Внешнее владение реальным email эта открытая demo-почта не доказывает.

PUT /candidate/profile — полная замена редактируемых полей ProfileInput, не PATCH. Клиент задаёт claimed_grade, а не confirmed_grade, skills или test_score. industry=it; специализации backend/frontend/qa, грейды junior/middle, формат remote/hybrid/office. Строгие целые зарплаты указаны в RUB. Резюме содержит full_name, years_experience, roles, soft_skills, education, about, declared_skills; заполнение не подтверждает компетентность. resume.declared_skills — до 20 ID технологий из declared_skills ответа /catalog, по умолчанию []. Повторы удаляются с сохранением порядка, неизвестный ID даёт 422. При полном PUT передавайте текущий список, чтобы сохранить заявления. React/Vue/Go/Java можно заявить, но оценочная категория остаётся одной из трёх доступных специализаций; заявление не обновляет skills, балл или публикацию. Смена специализации снимает публикацию и переключает подтверждённый результат на историю выбранного направления, не переносит квалификацию из другого.

POST /candidate/resume-preview принимает **сам PDF в теле**, Content-Type: application/pdf, до 2 MiB. Это не multipart/form-data. Ответ ResumePreviewView предлагает resume, contact, specialization, claimed_grade, claimed_skills, warnings, demo. Предпросмотр ничего не сохраняет: пользователь подтверждает поля отдельным PUT профиля. Парсинг эвристический, для текстового PDF, без OCR; ошибки/timeout — 422, слишком большой файл — 413, неправильный Content-Type — 415.

Каталог и основания навыков

GET /catalog возвращает CatalogView: industry, specializations, grades, assessment_version, declared_skills, work_tasks, work_tasks_version, demo. Сейчас assessment_version=scenarios-v6, work_tasks_version=team-work-v1. Элемент declared_skills содержит id/label/specializations, элемент work_tasks — id/label/description/skills/specialization. Каталог технологий для заявлений шире матрицы первичной оценки.

skill_evidence — SkillEvidenceView в собственном профиле и разрешённых работодателю проекциях кандидата (поиск, shortlist, detail). Источники разделены:

Поле: declared

Состав и смысл: ID заявленных технологий; без подтверждения навыка

Поле: assessment

Состав и смысл: skill, nullable score/correct/total/version/assessed_at, source=assessment либо legacy-assessment

Поле: practice

Состав и смысл: skill/score/tasks/total_tasks/completed_at/source; source — platform-practice либо employer-practice

Поле: limitations

Состав и смысл: Явные ограничения оснований

Основание оценки относится к текущему прошедшему результату категории, а не к последней неуспешной попытке. Без однозначного сохранённого результата используется legacy-assessment: баллы, количество и версия равны null. Сохранённая дата профиля может оставаться доступной.

Практика включает полностью завершённые и оценённые задания с заданным навыком. tasks считает задания с результатом ≥50%, total_tasks — все оценённые задания; score взвешен по весам заданий навыка. Платформенные и работодательские результаты агрегируются отдельно. Pending review/code, черновик или общая ручная оценка короткого ответа без навыков заданий не становятся доказательством конкретного навыка. Агрегат не выдаёт ответы, код, ключи, названия банков или контакты. Практика и заявления не повышают глобальный грейд и не участвуют в обязательном фильтре поиска.

Обязательная оценка, категория и публикация

Это первичная оценка ТЗ, отличная от практических банков работодателя. /catalog даёт актуальный набор навыков для каждой специализации. Новая форма содержит 12 вопросов из 12 семейств, четыре на каждый из трёх навыков. Сценарии включают чтение кода, реальные SQLite-таблицы, HTTP-повторы и права, HTML-доступность и QA-граничные случаи. Код кандидата этот API не исполняет.

1. В профиле выбрать специализацию и заявленный грейд.

2. POST /assessments/attempts: specialization, grade, опционально vacancy_id подходящей опубликованной вакансии. Ответ 201 AttemptView: id, questions, grade, specialization, context, version, result, answers, revision, expires_at, demo. Публичные вопросы исключают решения. Новые формы используют scenarios-v6. Публичная проекция вопроса: id/skill/title/options; family и parameters не возвращаются. ID новых вопросов — непрозрачные 24 hex-символа из сохранённого снимка; исторические ID сохранены для совместимости. Seed, answer, explanation, explanation_answer и topic не выдаются в вопросах. Старые формы v1/v2/v3/v4/v5 сохраняют свою версию и решения на сервере, новая версия их не пересчитывает.

У AttemptView добавлен nullable feedback: до завершения попытки всегда null, в том числе у истёкшего черновика. После успешной отправки владелец может запросить свою попытку через GET /assessments/attempts/{id} и получить разбор сохранённой формы: threshold, correct, total, needed_correct, skills и questions. needed_correct — общее минимальное число верных ответов для порога (9/12 Junior, 10/12 Middle), а не число недостающих. skills содержит correct/total/score, учебные темы и рекомендацию; status=strong начинается с 75% в этом навыке и не является новым грейдом. Каждый элемент questions содержит выбранный ответ, эталон и объяснение, со статусом correct, incorrect или unanswered. Для старых/неизвестных семейств подробное объяснение может быть недоступно — эталон снимка сохраняется. Работодатель получает 403, другой кандидат 404, анонимный запрос 401. Разбор не перезаписывает AssessmentResultView, историю, форму или баллы. Для v6 подробный разбор берётся из snapshot и показывается только при совпадении сохранённого ключа объяснения с эталоном; иначе используется честное сообщение о недоступном подробном разборе. Исторические формы без snapshot восстанавливаются своим прежним генератором.

3. PUT /assessments/attempts/{id}/draft передаёт **весь** словарь answers и expected_revision. Ответ — новая revision. ID вопросов и строковые варианты берутся из полученной формы. /candidate/active-assessment возвращает активную форму либо JSON null; GET попытки восстанавливает черновик.

4. POST .../submit принимает { "answers": {...} }, оценивает эти переданные ответы. Здесь нет revision и нет автоматической отправки сохранённого draft: передайте актуальный словарь явно. Клиентский score запрещён.
5. AssessmentResultView содержит score, skills, passed, version, limitation, demo, при наличии — breakdown. Повтор submit возвращает тот же сохранённый результат. Успех подтверждает категорию; провал сохраняет прежний подтверждённый результат. Форма действительна 60 минут.
6. POST /assessments/attempts/{id}/integrity принимает IntegrityBatch: от 1 до 100 событий {type,t,d?,q?}. Это технические сигналы открытой попытки, а не ответы: t — миллисекунды от её открытия, d — длительность возврата на вкладку, q разрешён только для ответа. Ответ 202 IntegrityAck содержит число сохранённых событий. Только владелец активной попытки может передать пакет; чужая или завершённая попытка не принимается. Ограничение — 60 запросов в минуту на вошедшего пользователя; при превышении возвращается 429 с Retry-After: 60. Счётчики ограничены сверху, переполнение становится сигналом и не отключает регистрацию последующих copy-событий.

В текущем Compose FSP_INTEGRITY_ENFORCE=1 по умолчанию: проходной балл при flagged сохраняется, но подтверждение категории удерживается. Разрешена одна немедленная перепроверка; после второго удержания следующая форма доступна через 14 суток. Клиент перед submit ждёт отправки очереди сигналов с ограниченным сетевым таймаутом. Это эвристики; отсутствие сигналов не доказывает отсутствие внешней помощи. Быстрые автоматические UX-прогоны явно используют enforcement=0, а удержание проверяется отдельными API-регрессиями. Сигналы и их пределы описаны в актуальное руководство (https://github.com/punkpsyh-hash/Fsp/blob/main/docs/50-completion-conformance.md); они не раскрывают текст ответа и при обычном demo-режиме сами по себе не блокируют грейд.

7. POST /candidate/consents с published:true и text_version=demo-consent-v1 включает поиск только после подтверждения. published:false снимает профиль с текущего поиска, shortlist и старых подборок. Уже выданный контакт управляется отдельным отзывом, см. ниже.

Повтор по тому же грейду/направлению — через 14 суток после submit; живая форма возвращается повторным start. Смена подтверждённого грейда

— не чаще 90 суток. Отсутствие подтверждения не компенсируется достижениями ФСП. История доступна через /candidate/assessment-history. Проверки сопоставимости на синтетике не являются независимой профессиональной аттестацией. Техническая проверка параметризации и устойчивости вариантов описана в актуальное руководство (https://github.com /punkpsyh-hash/Fsp/blob/main/docs/50-completion-conformance.md), включая сохранённые исторические v5 и текущую v6. Это не гарантия защиты от внешней помощи или равной сложности. Junior/Middle не калиброваны. Независимый пилот пока только пустой шаблон и анализатор в requirements/pilot/.

Подбор, предложение и контакт — основной сценарий ТЗ

Работодатель сохраняет свой CompanyInput и, при необходимости, NeedInput. Потребность содержит поля SearchInput плюс title/description. POST /company/searches принимает специализацию, подтверждённый грейд, must_skills, desired_skills, work_tasks, верхний бюджет salary_to, format, fsp_only. Ответ 201 SearchResultView содержит id, candidate_ids, rows, created_at, algorithm_version, demo. В строках — подтверждённые демо-оценкой навыки, skill_evidence, объяснение matched/missing/components и work_matches, итоговый score и fsp_bonus, а **не контакты**. Вес ранжирования: оценка 45%, навыки 35%, ФСП 10%, актуальность 10%.

SearchInput.work_tasks — до шести ID задач выбранной специализации из каталога team-work-v1, по умолчанию []. Повторы удаляются; неизвестный ID или задача другой специализации дают 422. Например, Backend: ["api-development", "sql-reporting"]. Связанные навыки добавляются к желательному совпадению, без изменения переданных must_skills/desired_skills и жёстких фильтров. Заявленные технологии и практика не подменяют их.

Элемент WorkTaskMatchView в work_matches содержит id/label/required_skills/matched_skills/missing_skills/status. status=covered означает отсутствие пробелов в ограниченной оценочной матрице; partial — есть совпадения и пробелы; gap — совпадений нет. Это объяснение соответствия навыков, а не подтверждение рабочего опыта. При выборе задач algorithm_version=weighted-v1+team-work-v1, иначе weighted-v1. Выбор

сохраняется в потребности и query истории поиска; объяснение work_matches сохраняется вместе с исходными баллами. Legacy snapshot без новых полей читается без выдуманного исторического объяснения.

GET /company/searches даёт последние десять собственных поисков. GET /company/searches/page?offset=0&limit=8 возвращает SearchHistoryPage: items (те же SearchSnapshotView), total, offset, limit. offset в диапазоне 0–2 147 483 647, limit 1–100, по умолчанию 8. Отрицательные/некорректные параметры — 422; страницы за концом списка возвращают пустые items с текущим total. Доступ только работодателю к своей истории; порядок — created_at DESC, id DESC. При новых поисках между запросами offset-страницы могут сдвинуться; интерфейс после создания подборки открывает первую страницу. Прежний list-маршрут сохраняется для совместимости. GET /company/searches/{search_id} сохраняет исторический порядок/объяснение, но проверяет текущую публикацию заново. historical_explanation:false означает legacy-снимок без исходных баллов; они не выдумываются. Shortlist принадлежит компании, операции PUT/DELETE возвращают 204.

POST /company/invitations принимает candidate_id, description, salary_from, salary_to, salary_basis (gross/net), необязательный vacancy_id. Зарплата — целое 1–10000000 RUB, верхняя граница не ниже нижней; описание 20–4000 символов. Профиль должен быть опубликован и подтверждён. Если передана вакансия, она должна принадлежать отправителю и быть опубликована. Обязательный заголовок Idempotency-Key — 8–128 символов: тот же ключ/тело возвращает прежнее предложение с 201; иное тело под тем же ключом — 409. Вакансия не обязательна: предложение компании непосредственно кандидату остаётся основным сценарием, а не витриной сторонних вакансий.

InvitationView включает компанию и способ связи, кандидата/alias, описание, зарплату и currency=RUB, status, expires_at, contact_revoked, demo. Поле contact необязательное и выдаётся работодателю при наличии его grant. Срок предложения — семь суток. Состояния: sent → viewed → accepted/declined; отправитель может перевести sent/viewed в withdrawn. Просроченная открытая запись показывается как expired; принять её нельзя. Повтор той же terminal decision идемпотентен, противоположная — 409.

После accepted только соответствующая компания получает контакт через свои приглашения, detail/PDF профиля. Другая компания не наследует доступ. DELETE /candidate/contacts/{company_id} возвращает 204 и отзывает доступ по всем приглашениям/откликам этому получателю. Повторное принятие прежнего accepted не отменяет отзыв. Публикация профиля и контактный grant различаются: выключение поиска само по себе не отзывает ранее выданный контакт. PDF компании строится из разрешённой проекции; ранее скачанные файлы сервер не может отозвать с компьютера получателя.

Вакансии, самостоятельный отклик и короткие задания

Собственная вакансия VacancyInput имеет draft/published/closed, специализацию/грейд, навыки, формат и обязательную зарплату. PUT требует актуальный expected_version; создание POST этого поля не требует. Чужая вакансия — 404, конфликт версии — 409. Кандидат видит опубликованные.

Добровольный POST /candidate/vacancies/{id}/apply требует подтверждённого грейда и сразу разрешает контакт **компании этой вакансии**. ApplicationView сохраняет snapshot вакансии, status, contact_revoked, даты и alias. Работодатель назначает viewed/accepted/declined; decline отзывает этот grant. Кандидат может сделать withdraw, повтор идемпотентен. Контакт может оставаться разрешённым по другому активному основанию той же компании; общий DELETE выше отзывает их все.

Короткие задания /company/tasks — отдельный простой формат текста с рубрикой, не constructor bank. Они предлагаются по подтверждённой специализации/грейду, не чаще раза в семь суток. Ответ 20–4000 символов; одинаковый повтор возвращает прежний ответ, изменение — 409. Автор оценивает 0–100 и оставляет feedback 20–2000 символов. Ручная оценка неизменна; score ≥70 обновляет activity_at, но не подтверждённый грейд или первичную оценку.

GET /candidate/tasks сохраняет available, next_at, responses и добавляет recommendation (либо null) и recommendation_state: offered, no-grade, cooldown, no-tasks. Рекомендация содержит id, публичное задание, начало недели UTC и начало следующей недели. Она формируется при открытии кабинета без записи в БД; учитывает подтверждённую категорию и

предыдущие ответы. Изменение набора доступных заданий может обновить выбор. Фоновой доставки и закреплённого назначения нет. Рекомендованное задание стоит первым в каталоге.

Конструктор, публикации и импорт

Предложение ИИ: POST /company/banks/{bank_id}/versions/{version_id}/suggest-task принимает revision, kind (четыре типа без code), specialization, grade, skills и brief (10–1500 символов). Параметры должны совпадать с текущим draft. Возвращает GenerationResult: авторский task, source, model, checks, review_required=true. Не сохраняет предложение и не публикует набор. Проверить эталон/рубрику, затем добавить в собственный draft через обычный PATCH. Только внутренний адрес модели; 10 запросов/мин на автора, максимум два одновременно. 503 — модель выключена/недоступна/занята, 502 — неверный ответ модели, 409 — старая или опубликованная версия, 429 — лимит. Чужая компания получает 404.

Автор видит только свои банки; scope platform доступен лишь локальному demo-куратору demo-employer-x, остальные используют employer. BankCreate: title, specialization, grade, time_limit_minutes, tasks, необязательный external_id, scope. Пустой draft допустим. BankView содержит current_version и versions, новые номера идут первыми.

PATCH версии передаёт **полный** документ и актуальную revision, а не набор частичных полей. Publish передаёт { "revision": N } и проверяет достаточность условий, навыков, ключей/рубрик/примеров. Ошибка полноты — 422, stale revision или правка published — 409. Published-content не редактируется: fork создаёт новый draft (201), archive снимает опубликованную версию с новой выдачи. Архив не удаляет существующие попытки. Preview выдаёт VersionPublic без answer_key, numeric_answer, hidden_tests. Автор получает VersionAuthor.

Тип: single_choice

Авторская проверка: ≥2 options, один ID в answer_key

Формат значения в answers: строка ID варианта

Тип: multiple_choice

Авторская проверка: ID вариантов, ключ; partial_credit

Формат значения в answers: массив различных ID

Тип: numeric

Авторская проверка: numeric_answer, absolute_tolerance, relative_tolerance, unit

Формат значения в answers: конечное число или строка с единицей

Тип: open_response

Авторская проверка: текстовая rubric, answer_limit

Формат значения в answers: строка; проверяет автор

Тип: code

Авторская проверка: language, starter_code, публичные examples, hidden_tests, runtime_limits

Формат значения в answers: строка исходного кода

Для multiple-choice частичный балл при включении: max(0, (верно выбранные − неверно выбранные) / количество ключей) × 100. Иначе нужен точный набор. Numeric использует максимальную абсолютную/относительную погрешность. Итог банка — взвешенное среднее по weight; незавершённые задачи пока вносят 0 в число score, поэтому ориентируйтесь также на completed/pending_count.

POST /company/bank-imports/dry-run?filename=... принимает raw bytes, рекомендуемый Content-Type: application/octet-stream. Расширение filename определяет парсер; multipart не используется. До 2 MiB, filename ≤200 символов. column_mapping — query-параметр с URL-encoded JSON объектом «исходная колонка → каноническая» только для CSV; пустое значение пропускает колонку. Неизвестные/повторные соответствия запрещены.

Ответ **200** ImportView: id, valid, errors, warnings, preview, duplicate_bank_id, publication_errors. Ошибка файла, включая неизвестное расширение, отражается valid:false/errors, не обязательно HTTP-ошибкой. Сам dry-run сохраняет историю проверки, но не создаёт банк. valid:true означает валидный draft, а не готовность публикации: проверяйте publication_errors. Неверный JSON mapping/параметры — 422; поток больше

лимита — 413. Разбор нового файла выполняется вне event loop: одновременно допускаются два импорта в процессе API, не более десяти за минуту на владельца. При исчерпании допуска или квоты — 429 и Retry-After: 60. Повтор своего уже проверенного файла использует cache без нового разбора. Лимиты PDF (CPU, память, страницы и timeout) сохраняются. POST .../{import_id}/confirm создаёт draft атомарно; чужой import — 404, ошибочный — 422; одинаковое подтверждение не создаёт второй банк/версию. Повторный документ для своего external_id создаёт следующую draft-версию. Файл, расширение и mapping входят в идемпотентность dry-run.

JSON/ZIP сохраняют полный авторский документ fsp-bank-v1. ZIP содержит manifest.json, опциональные текстовые starter-файлы, до 50 файлов/4 MiB распакованного размера; traversal, symlinks, шифрование и опасная упаковка запрещены. CSV — UTF-8/BOM, выборы/числа; PDF/DOCX извлекают лишь текст в неполный черновик, без придуманных навыков/ключей/рубрик. Текстовые PDF — до 30 страниц, без OCR. Полная схема, CSV-поля/экранирование формул и файлы: requirements/examples (../requirements/examples/README.md).

GET /company/banks/{id}/export?format=json|csv|zip экспортирует последнюю версию своего банка, включая авторские ключи. JSON → BankDocument, CSV → text/csv, ZIP → application/zip. CSV сложного/open/code банка — 422; используйте JSON/ZIP. Этот export не доступен кандидату.

Выполнение банка и очередь VM

Кандидатский GET /candidate/banks показывает последнюю опубликованную версию каждого банка. Start принимает version_id, возвращает одну попытку кандидата на эту версию с 201; повтор start возвращает прежнюю, не сбрасывает время. PATCH попытки передаёт весь answers плюс revision. Deadline определяется сервером из лимита версии; после срока новые ответы не принимаются. Submit принимает **только revision**, оценивает уже сохранённые ответы. После deadline submit также оценивает только ранее сохранённые данные. Это отличается от первичной оценки, где submit принимает answers.

AttemptPublic: ID/revision/status/deadline, неизменная кандидатская версия, свои answers, result, alias. Status: in_progress, pending_review, completed. AttemptResult: score, pending_count, completed, task-results, scope, **`grade_effect:false`**, scorer_version=bank-v1. Task-result имеет score:null и pending_review/pending_code, пока результата нет. Открытый ответ проверяет только владелец банка после submit; rubric доступна кандидату, ключи/hidden tests — нет. Ручной score 0–100, feedback 2–4000 символов; одинаковый повтор идемпотентен, изменение зафиксированной review — 409.

POST .../code-jobs принимает task_id/source. Ответ 201 CodeJobView: id, task_id, status, execution_status, result, error_code, даты. Status очереди: queued/running/completed/failed; execution-status: passed/wrong_answer/timeout/runtime_error/system_error. result содержит passed_tests, total_tests, score, runtime_version, duration_ms и status; stdout/fixtures/lease/source не выдаются кандидату. Статус запрашивается GET по job ID в своей попытке. Повтор источника возвращает имеющийся job, если он не failed. До submit — 10 предварительных jobs, после submit — не более 20 суммарно; публикация допускает максимум 10 code задач. Одновременно допускается один активный предварительный job на попытку. Финальный submit резервирует исполнения всех сохранённых code-ответов. После submit можно повторить лишь **точно тот же сохранённый source**.

Код исполняется отдельным Docker worker в одноразовой Linux VM, не в API. Языки python/javascript/sql; SQL — **SQLite**, один SELECT над fixture, не PostgreSQL. Python/JS получают stdin, stdout сравнивается с эталоном; SQL возвращает упорядоченные строки. VM фиксированно 512 MiB; memory_mb принимает только 512. Поле wall_seconds (1–3) исторически названо так, но задаёт CPU-бюджет KVM; явный TCG использует обозначенный множитель ×10. Код/шаблон ≤32768 UTF-8 bytes, вход ≤16384, setup ≤32768, вывод ≤64 KiB; скрытых cases ≤10 и суммарный JSON-бюджет ≤750000 bytes. Точные VM/host ограничения и версии — актуальное руководство (https://github.com/punkpsyh-hash/Fsp/blob/main/docs/44-deployment-and-operations.md).

Инфраструктурный сбой остаётся pending_code, а не неправильным ответом. Lease истекает через 900 секунд, предусмотрен один автоматический retry; затем допустим ограниченный повтор кандидатом с frozen source. Наличие runner-ключа проверяется при ручном enqueue;

отсутствие даёт 503. При остановленном worker уже созданный queued job ожидает восстановления.

Внутренние /internal/runner/* предназначены worker, не браузерному клиенту. Используют Authorization: Bearer <локально настроенный ключ>, не cookie/CSRF. Claim возвращает { "job": null } либо job с id, lease_token, language, source, tests (stdin/setup/expected), time_limit_seconds. Result принимает lease_token, status, passed, total, duration_ms, runtime_version и необязательный enum error_code. Истёкший/повторный lease — 409, несоответствие закреплённым cases — 422, неверная авторизация — 403, ненастроенный broker — 503. Полные внутренние поля доступны лишь worker.

Ошибки и форматы

Пользовательские входные схемы запрещают неизвестные поля. Строгие целые не принимают bool/float/строковые числа. Время — ISO 8601 UTC. Успешные статусы и схемы каждой операции — в таблице ниже; 204 имеет пустое тело. JSON ответы API идут с Cache-Control: no-store; обычные ответы содержат X-Request-ID. Middleware-ошибка может содержать request_id только в JSON.

Пример · json

```
{"code":"http_409","message":"Ответы изменены в другой вклад
ке; загрузите сохранённую попытку","request_id":"<id запроса
>"}
```

Это пример формы ошибки; message зависит от конкретного обработчика. Ошибка валидации имеет code=validation, message, request_id и details с field/message. Origin даёт origin_forbidden, auth quota — rate_limit.

Статус: 401

Практический смысл: нет действующей сессии/подтверждения; неправильный пароль

Статус: 403

Практический смысл: роль, CSRF, Origin или worker Bearer не разрешены

Статус: 404

Практический смысл: отсутствующая/чужая запись либо неподходящая опубликованная сущность

Статус: 409

Практический смысл: конфликт revision/version, terminal state, дедлайн/cooldown или idempotency

Статус: 413

Практический смысл: raw PDF/import превышает 2 MiB

Статус: 415

Практический смысл: resume-preview требует Content-Type application/pdf

Статус: 422

Практический смысл: поля/ответы/mapping невалидны; публикация не готова; invalid import confirm; CSV export сложных типов

Статус: 429

Практический смысл: quota входа или лимит code jobs

Статус: 503

Практический смысл: компонент не настроен/недоступен в явно предусмотренной обработчиком ветке

OpenAPI описывает общий набор возможных ошибок для операций; это не означает, что каждый путь обязательно имеет каждую ветку. Ошибки файла dry-run обычно возвращаются внутри успешного ImportView. При 409 сначала получите серверную версию и согласуйте свою правку; слепой повтор с новой revision теряет смысл CAS.

ФСП ID mock и исходное ТЗ

ТЗ §2 и §4 намеренно не предоставляет внешний API/структуру данных ФСП; Keycloak — ориентир совместимости, не обязательная авторизация MVP. Здесь реализован **локальный** OIDC mock authorization code + PKCE S256, а не внешний вход. Путь кандидата: с пользовательской сессией открыть GET /candidate/fsp-id/start (302) → пройти HTML-форму mock ID → callback привяжет synthetic sub и загрузит demo-события. Discovery доступен локально по /auth/mock-fsp-id/.well-known/openid-configuration; authorize принимает только заранее созданные state, client_id и callback, token принимает FspToken (grant_type=authorization_code, client_id, code, code_verifier) и возвращает FspTokenOut, без access token.

После callback профиль получает none, linked-empty, verified или unavailable; отсутствие истории не блокирует оценку или публикацию. DELETE /candidate/fsp-link отвязывает mock ID и возвращает FspLinkView (state, previous_state, signal, demo). Member ID не выдаётся работодателю в поиске. Этот mock не проверяет внешнюю личность и не присваивает подтверждённый грейд.

Официальный публичный OIDC discovery ФСП ID исследован отдельно: актуальное руководство (https://github.com/punkpsyh-hash/Fsp/blob/main/docs/50-completion-conformance.md). Сам факт issuer/PKCE не регистрирует наш client и не разрешает получать достижения. В этом сервисе нет внешнего OIDC callback, внешнего token exchange/userinfo или API реестра. Новые внешние пути/поля до получения официального контракта не предполагаются.

Полный справочник операций

Таблица сверена с generated OpenAPI. [] означает массив выходной схемы, null — отсутствие активной формы. Схемы входа — JSON, кроме отмеченных raw файлов; — означает отсутствие JSON body. Path ID, query и заголовки описаны выше и полностью заданы OpenAPI. K — кандидат, P — работодатель, П — публично, C — любая пользовательская сессия, W — внутренний worker. Все операции P над банками/ответами/подборками проверяют соответствующее владение; операции K над попытками/приглашениями/откликами — собственную запись.

Состояние

GET /api/health

Роль / назначение: П — Проверка API и соединения БД

Вход → статус / ответ: — → 200 / HealthView

Mock-идентификация

GET /api/v1/auth/demo-accounts

Роль / назначение: П — Список seed-аккаунтов

Вход → статус / ответ: — → 200 / AccountView[]

POST /api/v1/auth/demo-login

Роль / назначение: П — Вход в seed-аккаунт

Вход → статус / ответ: DemoLogin → 200 / SessionView

POST /api/v1/auth/register

Роль / назначение: П — Регистрация синтетического адреса

Вход → статус / ответ: Register → 201 / DeliveryView

POST /api/v1/auth/login

Роль / назначение: П — Повторная ссылка входа

Вход → статус / ответ: Email → 200 / MessageView

POST /api/v1/auth/verify

Роль / назначение: П — Одноразовое подтверждение адреса

Вход → статус / ответ: Verify → 200 / SessionView

Вход

POST /api/v1/auth/password-login

Роль / назначение: П — Вход с подтверждённым адресом/паролем

Вход → статус / ответ: PasswordLogin → 200 / SessionView

POST /api/v1/auth/jury-register

Роль / назначение: П — Индивидуальный синтетический аккаунт, только opt-in jury

Вход → статус / ответ: JuryRegister → 201 / JurySession

Mock-почта

GET /api/v1/auth/mock-mail

Роль / назначение: П — Локальная demo-почта

Вход → статус / ответ: — → 200 / MailView[]

Сессия

GET /api/v1/me

Роль / назначение: С — Своя сессия и CSRF

Вход → статус / ответ: — → 200 / SessionView

POST /api/v1/auth/logout

Роль / назначение: С — Завершить свою сессию

Вход → статус / ответ: — → 204 / —

Профиль

GET /api/v1/candidate/profile

Роль / назначение: К — Полный собственный профиль

Вход → статус / ответ: — → 200 / CandidateSelfView

PUT /api/v1/candidate/profile

Роль / назначение: К — Заменить редактируемые поля

Вход → статус / ответ: ProfileInput → 200 / SavedView

POST /api/v1/candidate/resume-preview

Роль / назначение: K — Предпросмотр raw PDF без сохранения

Вход → статус / ответ: raw application/pdf → 200 / ResumePreviewView

Приватность

POST /api/v1/candidate/consents

Роль / назначение: K — Включить/отозвать публикацию

Вход → статус / ответ: ConsentInput → 200 / PublicationView

DELETE /api/v1/candidate/contacts/{company_id}

Роль / назначение: K — Отозвать контакт компании

Вход → статус / ответ: — → 204 / —

Оценка

GET /api/v1/candidate/active-assessment

Роль / назначение: K — Активная первичная форма

Вход → статус / ответ: — → 200 / AttemptView / null

PUT /api/v1/assessments/attempts/{attempt_id}/draft

Роль / назначение: K — Сохранить draft с CAS

Вход → статус / ответ: DraftAnswers → 200 / AttemptView

GET /api/v1/candidate/assessment-history

Роль / назначение: K — Собственная история оценок/грейда

Вход → статус / ответ: — → 200 / AssessmentHistoryView

Демо-оценка

POST /api/v1/assessments/attempts

Роль / назначение: K — Начать/вернуть первичную форму

Вход → статус / ответ: AttemptInput → 201 / AttemptView

GET /api/v1/assessments/attempts/{attempt_id}

Роль / назначение: K — Своя форма/черновик

Вход → статус / ответ: — → 200 / AttemptView

POST /api/v1/assessments/attempts/{attempt_id}/submit

Роль / назначение: К — Оценить переданные answers

Вход → статус / ответ: Answers → 200 / AssessmentResultView

POST /api/v1/assessments/attempts/{attempt_id}/integrity

Роль / назначение: К — Передать пакет технических сигналов своей активной формы

Вход → статус / ответ: IntegrityBatch → 202 / IntegrityAck

ФСП ID mock

GET /api/v1/auth/mock-fsp-id/.well-known/openid-configuration

Роль / назначение: П — Discovery локального OIDC mock

Вход → статус / ответ: — → 200 / object

GET /api/v1/candidate/fsp-id/start

Роль / назначение: К — Создать state/PKCE и перенаправить к форме mock ID

Вход → статус / ответ: — → 302 / HTML authorize

GET /api/v1/auth/mock-fsp-id/authorize

Роль / назначение: П — HTML-форма mock ID для валидного state

Вход → статус / ответ: query state,redirect_uri,client_id → 200 HTML / 400 HTML

POST /api/v1/auth/mock-fsp-id/authorize

Роль / назначение: П — Проверить ID/пароль и вернуть одноразовый code

Вход → статус / ответ: form-urlencoded → 302 callback / 400 HTML

POST /api/v1/auth/mock-fsp-id/token

Роль / назначение: П — Обмен code и verifier локального mock

Вход → статус / ответ: FspToken → 200 / FspTokenOut

GET /api/v1/candidate/fsp-id/callback

Роль / назначение: К — Закончить привязку только к начавшей сессии

Вход → статус / ответ: query code,state → 302 privacy

DELETE /api/v1/candidate/fsp-link

Роль / назначение: K — Отвязать свой ФСП ID mock

Вход → статус / ответ: — → 200 / FspLinkView

Подбор

GET /api/v1/company/searches

Роль / назначение: P — Последние собственные поиски

Вход → статус / ответ: — → 200 / SearchSnapshotView[]

GET /api/v1/company/searches/page

Роль / назначение: P — Страница полной собственной истории; offset/limit

Вход → статус / ответ: — → 200 / SearchHistoryPage

POST /api/v1/company/searches

Роль / назначение: P — Новый поиск и snapshot

Вход → статус / ответ: SearchInput → 201 / SearchResultView

POST
/api/v1/company/searches/{search_id}/candidates/{candidate_id}/explanation

Роль / назначение: P — Словесное объяснение строки своей подборки

Вход → статус / ответ: — → 200 / ExplanationOut

GET /api/v1/company/candidates/{candidate_id}

Роль / назначение: P — Разрешённая проекция кандидата

Вход → статус / ответ: — → 200 / CompanyCandidateView

GET /api/v1/company/shortlist

Роль / назначение: P — Свой shortlist

Вход → статус / ответ: — → 200 / CandidateView[]

PUT /api/v1/company/shortlist/{candidate_id}

Роль / назначение: P — Добавить опубликованный профиль

Вход → статус / ответ: — → 204 / —

DELETE /api/v1/company/shortlist/{candidate_id}

Роль / назначение: P — Удалить из своего shortlist

Вход → статус / ответ: — → 204 / —

GET /api/v1/company/categories

Роль / назначение: P — Опубликованные категории/численность

Вход → статус / ответ: — → 200 / CategoryView[]

GET /api/v1/company/searches/{search_id}

Роль / назначение: P — Своя историческая подборка с текущими правами

Вход → статус / ответ: — → 200 / RestoredSearchView

Приглашения

POST /api/v1/company/invitations

Роль / назначение: P — Предложение с зарплатой/idempotency

Вход → статус / ответ: OfferInput → 201 / InvitationView

GET /api/v1/company/invitations

Роль / назначение: P — Свои отправленные предложения

Вход → статус / ответ: — → 200 / InvitationView[]

GET /api/v1/candidate/invitations

Роль / назначение: K — Свои полученные предложения

Вход → статус / ответ: — → 200 / InvitationView[]

POST /api/v1/candidate/invitations/{offer_id}/view

Роль / назначение: K — Отметить своё предложение просмотренным

Вход → статус / ответ: — → 200 / InvitationView

POST /api/v1/candidate/invitations/{offer_id}/decision

Роль / назначение: K — Принять/отклонить своё предложение

Вход → статус / ответ: Decision → 200 / InvitationView

POST /api/v1/company/invitations/{offer_id}/withdraw

Роль / назначение: P — Отозвать своё открытое предложение

Вход → статус / ответ: — → 200 / InvitationView

Сообщения

Диалог доступен только при открытом контакте (принятое приглашение либо активный отклик); после отзыва контакта история читается, отправка возвращает 403. Лимит — 20 сообщений в час на отправителя в диалоге (429). Текст хранится как обычный текст, длина 1–2000 символов. Чтение не меняет данные; прочитанность отмечает отдельный POST …/read.

GET /api/v1/company/chats

Роль / назначение: P — Свои диалоги с кандидатами

Вход → статус / ответ: — → 200 / ChatThreadView[]

GET /api/v1/company/chats/{candidate_id}/messages

Роль / назначение: P — Сообщения диалога

Вход → статус / ответ: limit → 200 / ChatMessageView[]

POST /api/v1/company/chats/{candidate_id}/messages

Роль / назначение: P — Отправить сообщение

Вход → статус / ответ: ChatMessageInput → 201 / ChatMessageView

POST /api/v1/company/chats/{candidate_id}/read

Роль / назначение: P — Отметить входящие прочитанными

Вход → статус / ответ: — → 200 / ReadResult

GET /api/v1/candidate/chats

Роль / назначение: K — Свои диалоги с компаниями

Вход → статус / ответ: — → 200 / ChatThreadView[]

GET /api/v1/candidate/chats/{company_id}/messages

Роль / назначение: K — Сообщения диалога

Вход → статус / ответ: limit → 200 / ChatMessageView[]

POST /api/v1/candidate/chats/{company_id}/messages

Роль / назначение: K — Отправить сообщение

Вход → статус / ответ: ChatMessageInput → 201 / ChatMessageView

POST /api/v1/candidate/chats/{company_id}/read

Роль / назначение: К — Отметить входящие прочитанными

Вход → статус / ответ: — → 200 / ReadResult

Экспорт

GET /api/v1/candidate/profile.pdf

Роль / назначение: К — PDF своего профиля

Вход → статус / ответ: — → 200 / application/pdf

GET /api/v1/company/candidates/{candidate_id}/profile.pdf

Роль / назначение: Р — PDF разрешённой проекции

Вход → статус / ответ: — → 200 / application/pdf

Справочники

GET /api/v1/catalog

Роль / назначение: П — Направления/грейды/навыки

Вход → статус / ответ: — → 200 / CatalogView

Компания

GET /api/v1/company/profile

Роль / назначение: Р — Собственный профиль компании

Вход → статус / ответ: — → 200 / CompanyView

PUT /api/v1/company/profile

Роль / назначение: Р — Сохранить профиль компании

Вход → статус / ответ: CompanyInput → 200 / CompanyView

Потребности

GET /api/v1/company/needs

Роль / назначение: Р — Сохранённые собственные потребности

Вход → статус / ответ: — → 200 / NeedView[]

POST /api/v1/company/needs

Роль / назначение: Р — Создать собственную потребность

Вход → статус / ответ: NeedInput → 201 / NeedView

Вакансии

GET /api/v1/company/vacancies

Роль / назначение: P — Свои вакансии

Вход → статус / ответ: — → 200 / VacancyView[]

POST /api/v1/company/vacancies

Роль / назначение: P — Создать свою вакансию

Вход → статус / ответ: VacancyInput → 201 / VacancyView

PUT /api/v1/company/vacancies/{vacancy_id}

Роль / назначение: P — Заменить свою вакансию с version CAS

Вход → статус / ответ: VacancyInput → 200 / VacancyView

GET /api/v1/candidate/vacancies

Роль / назначение: K — Опубликованные вакансии

Вход → статус / ответ: — → 200 / VacancyView[]

Отклики

POST /api/v1/candidate/vacancies/{vacancy_id}/apply

Роль / назначение: K — Отклик и grant компании вакансии

Вход → статус / ответ: — → 201 / ApplicationView

GET /api/v1/candidate/applications

Роль / назначение: K — Свои отклики

Вход → статус / ответ: — → 200 / ApplicationView[]

GET /api/v1/company/applications

Роль / назначение: P — Отклики на свои вакансии

Вход → статус / ответ: — → 200 / ApplicationView[]

POST /api/v1/company/applications/{application_id}/decision

Роль / назначение: P — Решение по отклику своей компании

Вход → статус / ответ: ApplicationDecision → 200 / ApplicationView

POST /api/v1/candidate/applications/{application_id}/withdraw

Роль / назначение: K — Отозвать свой отклик

Вход → статус / ответ: — → 200 / ApplicationView

Короткие задания

GET /api/v1/company/tasks

Роль / назначение: P — Свои короткие текстовые задания

Вход → статус / ответ: — → 200 / EmployerTaskView[]

POST /api/v1/company/tasks

Роль / назначение: P — Создать короткое задание

Вход → статус / ответ: TaskInput → 201 / EmployerTaskView

GET /api/v1/candidate/tasks

Роль / назначение: K — Доступные короткие задания/свои ответы

Вход → статус / ответ: — → 200 / SuggestedTasksView

POST /api/v1/candidate/tasks/{task_id}/submit

Роль / назначение: K — Ответить на доступное короткое задание

Вход → статус / ответ: TaskAnswer → 201 / TaskResponseView

GET /api/v1/company/task-responses

Роль / назначение: P — Ответы на свои короткие задания

Вход → статус / ответ: — → 200 / TaskResponseView[]

POST /api/v1/company/task-responses/{response_id}/review

Роль / назначение: P — Зафиксировать ручную оценку своего задания

Вход → статус / ответ: TaskReview → 200 / TaskResponseView

Конструктор

GET /api/v1/company/banks

Роль / назначение: P — Свои банки со всеми версиями

Вход → статус / ответ: — → 200 / BankView[]

POST /api/v1/company/banks

Роль / назначение: P — Создать банк и draft

Вход → статус / ответ: BankCreate → 201 / BankView

GET /api/v1/company/banks/{bank_id}

Роль / назначение: P — Свой банк и история версий

Вход → статус / ответ: — → 200 / BankView

PATCH /api/v1/company/banks/{bank_id}/versions/{version_id}

Роль / назначение: P — Заменить свой draft с revision CAS

Вход → статус / ответ: VersionSave → 200 / VersionAuthor

POST /api/v1/company/banks/{bank_id}/versions/{version_id}/suggest-task

Роль / назначение: P — Предложить приватный ИИ-черновик для своей текущей draft-версии; без записи и публикации

Вход → статус / ответ: TaskSuggestionInput → 200 / GenerationResult

POST /api/v1/company/banks/{bank_id}/versions/{version_id}/publish

Роль / назначение: P — Проверить и опубликовать draft

Вход → статус / ответ: Revision → 200 / VersionAuthor

POST /api/v1/company/banks/{bank_id}/versions/{version_id}/fork

Роль / назначение: P — Создать draft из своей версии

Вход → статус / ответ: — → 201 / VersionAuthor

POST /api/v1/company/banks/{bank_id}/versions/{version_id}/archive

Роль / назначение: P — Архивировать свою опубликованную версию

Вход → статус / ответ: — → 200 / VersionAuthor

GET /api/v1/company/banks/{bank_id}/versions/{version_id}/preview

Роль / назначение: P — Кандидатский preview своей версии

Вход → статус / ответ: — → 200 / VersionPublic

Импорт заданий

GET /api/v1/company/banks/{bank_id}/export

Роль / назначение: P — Авторский export последней версии

Вход → статус / ответ: — → 200 / BankDocument, text/csv, application/zip

POST /api/v1/company/bank-imports/dry-run

Роль / назначение: P — Raw dry-run и история проверки

Вход → статус / ответ: raw application/octet-stream → 200 / ImportView

POST /api/v1/company/bank-imports/{import_id}/confirm

Роль / назначение: P — Атомарно подтвердить свой импорт

Вход → статус / ответ: — → 200 / BankView

Выполнение заданий

GET /api/v1/candidate/banks

Роль / назначение: K — Последние опубликованные версии банков

Вход → статус / ответ: — → 200 / VersionPublic[]

GET /api/v1/candidate/bank-attempts

Роль / назначение: K — История собственных практик

Вход → статус / ответ: — → 200 / AttemptPublic[]

POST /api/v1/candidate/banks/{bank_id}/attempts

Роль / назначение: K — Начать/вернуть свою практику версии

Вход → статус / ответ: AttemptStart → 201 / AttemptPublic

GET /api/v1/candidate/bank-attempts/{attempt_id}

Роль / назначение: K — Своя практика/answers/result

Вход → статус / ответ: — → 200 / AttemptPublic

PATCH /api/v1/candidate/bank-attempts/{attempt_id}

Роль / назначение: K — Полное autosave с revision CAS

Вход → статус / ответ: AnswersSave → 200 / AttemptPublic

POST /api/v1/candidate/bank-attempts/{attempt_id}/submit

Роль / назначение: K — Отправить сохранённые answers

Вход → статус / ответ: Revision → 200 / AttemptPublic

Выполнение кода

POST /api/v1/candidate/bank-attempts/{attempt_id}/code-jobs

Роль / назначение: K — Создать/вернуть VM job своего source

Вход → статус / ответ: CodeSubmit → 201 / CodeJobView

GET /api/v1/candidate/bank-attempts/{attempt_id}/code-jobs/{job_id}

Роль / назначение: K — Статус своего VM job

Вход → статус / ответ: — → 200 / CodeJobView

Ручная проверка

GET /api/v1/company/banks/{bank_id}/attempts

Роль / назначение: P — Отправленные попытки своего банка

Вход → статус / ответ: — → 200 / AttemptAuthor[]

POST /api/v1/company/bank-attempts/{attempt_id}/review-suggestion

Роль / назначение: P — ИИ-подсказка оценки открытого ответа (ничего не сохраняет; нужна ручная фиксация)

Вход → статус / ответ: ReviewSuggestionInput → 200 / ReviewSuggestion; 422 не открытый ответ; 503 ИИ выключен или модель недоступна; 502 ответ не прошёл проверку; 429 лимит

POST /api/v1/company/bank-attempts/{attempt_id}/review

Роль / назначение: P — Зафиксировать review своего открытого задания

Вход → статус / ответ: ReviewInput → 200 / AttemptAuthor

Внутренний runner

POST /internal/runner/claim

Роль / назначение: W — Получить job/lease или null

Вход → статус / ответ: — → 200 / RunnerClaim

POST /internal/runner/jobs/{job_id}/result

Роль / назначение: W — Зафиксировать leased результат исполнения

Вход → статус / ответ: RunnerResult → 200 / Saved

Историческая проверка руководства (72 пути / 82 операции)

Повторно прочитан девятистраничный оригинал ТЗ; сверены app/schemas/contract, product, banks, security и тестовые сценарии API. Файл auth.py отсутствует: сессии/CSRF находятся в app.py, пароли — security.py. Справочник получен из OpenAPI с ручной проверкой ролей/назначений: все 82 method/path представлены ровно один раз, лишних операций нет. Offline-проверка файла HTTP подтвердила 62 запроса к действительным операциям; все 30 JSON-тел сверены с входными схемами после безопасной подстановки синтетических значений. Защищённые внешние ФСП вызовы не выполнялись. Полная действующая regression-приёмка — актуальное руководство (https://github.com/punkpsyh-hash/Fsp/blob/main/docs/48-release-acceptance.md); этот справочник не обещает новый прогон всего приложения или production-готовность.

Дополнение 10 октября (сообщения) довело контракт до 86 путей / 99 операций; дополнение 9 октября содержало 80 путей / 91 операцию, включая добросовестность, локальный OIDC mock, предложение ИИ-черновика и рекомендацию короткого задания на неделю. Текущая проверка — polish-api-results.json (архив этапа в закрытой истории), PDF — polish-api-pdf-results.json (архив этапа в закрытой истории); протокол доводки — polish-results.json (архив этапа в закрытой истории) и актуальное руководство (https://github.com/punkpsyh-hash/Fsp/blob/main/docs/45-ai-answer-check-and-training-package.md). Прежние docs/final-api-results.json, final-api-pdf-results.json (архив этапа в закрытой истории) и актуальное руководство (https://github.com/punkpsyh-hash/Fsp/blob/main/docs/48-release-acceptance.md) относятся к этапу 79 путей / 90 операций.

Иллюстрация пользовательского контроля доступа

На рисунке 1 показаны настройки кандидата на работающем синтетическом стенде. Они поясняют назначение операций приватности; проверку полномочий выполняет сервер. Скрытие элементов интерфейса само по себе не заменяет авторизацию API.

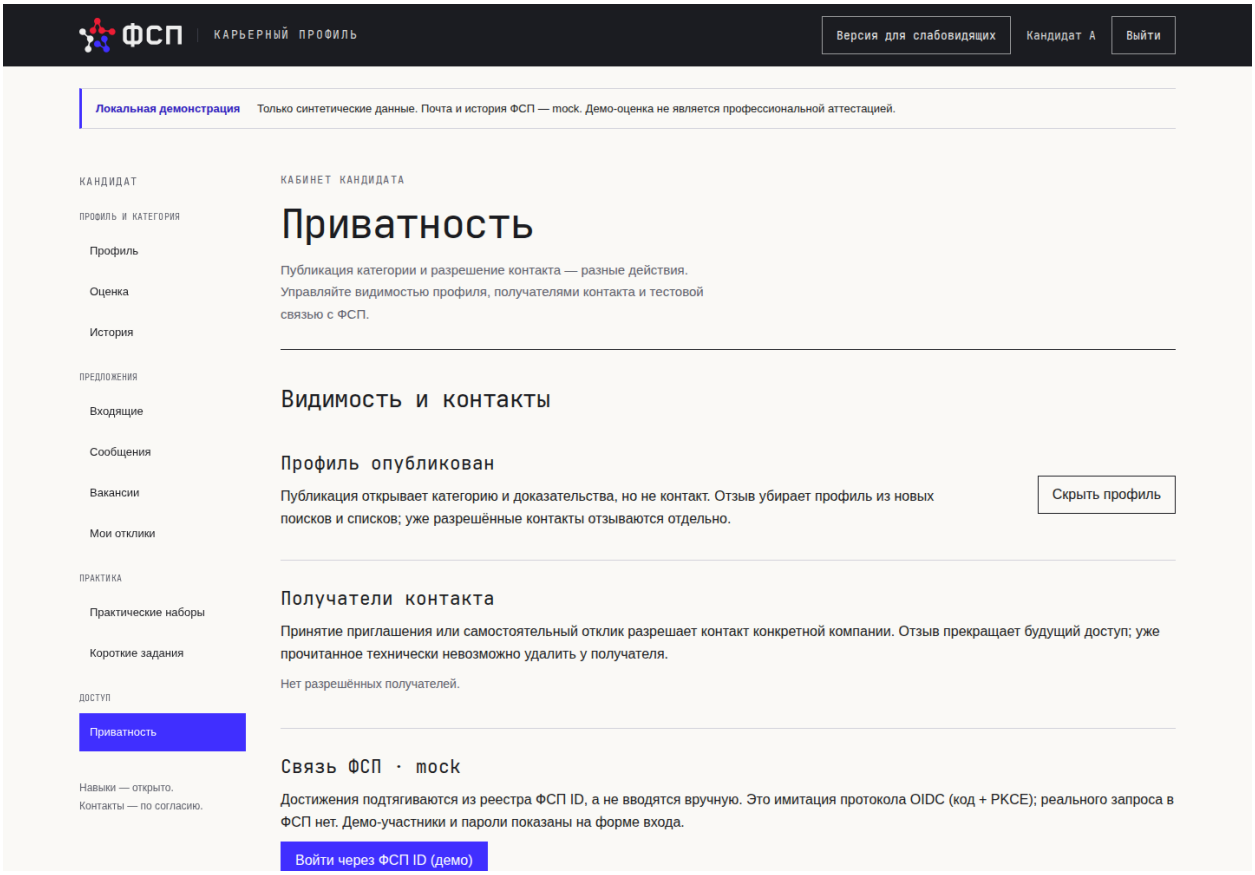


Рисунок 1 — Настройки доступности профиля и контактов

Источник: Работающий синтетический стенд NAS, Chromium; 11 октября 2026 года